8 Alternative JVM languages

This chapter covers

- Language zoology
- Why you should use alternative JVM languages
- Selection criteria for alternative languages
- How the JVM handles alternative languages

If you've used Java for any sizable amount of work, you've probably noticed that it can tend toward being a bit verbose and clumsy at times. You may even have found yourself wishing that things were different—easier somehow.

Fortunately, as you've seen in the last few chapters, the JVM is awesome—so awesome, in fact, that it provides a natural home for programming languages other than Java. In this chapter, we'll show you why and how you might want to start mixing another JVM programming language into your project.

In this chapter, we'll cover ways of describing the different language types (such as static versus dynamic typing), why you might want to use alternative languages, and what criteria to look for in choosing them. You'll also be introduced to the two languages (Kotlin and Clojure) that we'll cover in more depth throughout the remainder of this book.

8.1 Language zoology

Programming languages come in many different flavors and classifications. Another way of saying this is that a wide range of styles and approaches to programming are embodied in different languages. Mastering these different styles is often easier when you understand how to classify the differences between languages.

NOTE These classifications are an aid to thinking about the diversity of languages. Some of these divisions are more clear-cut than others, but none of the classifying schemes is perfect. Different people have different ideas about how the classification should be laid out.

In recent years, a trend has existed for languages to add features from across the spectrum of possibilities. It's often helpful to think of a given language as being "less functional" than another language, or "dynamically typed but with optional static typing when needed."

The classifications we'll cover are "interpreted versus compiled," "dynamic versus static," "imperative versus functional," and reimplementations of a language versus the original. In general, these classifications are tools for thinking about the space, rather than complete and precise academic schemes.

For example, we can say Java is a runtime-compiled, statically typed, imperative language with some functional features. It emphasizes safety, code clarity, backward compatibility, and performance, and it's happy to accept a certain amount of verbosity and *ceremony* (such as in deployment) to achieve those goals.

NOTE Different languages may have different priorities; for example, dynamically typed languages may emphasize deployment speed.

Let's get started with the interpreted versus compiled classification.

1.1.1 Interpreted vs. compiled languages

An *interpreted language* is one in which each step of the source code is executed as is, rather than the entire program being transformed to machine code before execution begins. This contrasts with a compiled language, which is one that uses a compiler to convert the human-readable source code into a binary form as an initial task.

This distinction has become less clear recently. In the early '90s, the divide was fairly clear: C/C++, FORTRAN, and their friends were compiled languages, and Perl and Python were interpreted languages. But as we alluded to in chapter 1, Java has features of both compiled and interpreted languages. The use of bytecode further muddies the issue. Bytecode is certainly not human readable, but neither is it machine code.

For the JVM languages we'll study in this part of the book, the distinction we'll make is whether the language produces a class file from the source code and executes that—or not. In the latter case, an interpreter (probably written in Java) is used to execute the source code, line by line. Some languages provide both a compiler and an interpreter, and some provide an interpreter and a just-in-time (JIT) compiler that will emit JVM bytecode.

1.2 Dynamic vs. static typing

In languages with dynamic typing, a variable can contain different types at different times during a program's execution. As an example, let's look at a simple bit of code in a well-known dynamic language, JavaScript. This example should hopefully be comprehensible, even if you don't know the language in detail:

```
var answer = 40;
answer = answer + 2;
answer = "What is the answer? " + answer;
```

The `var` keyword used here creates a new variable. In JavaScript's dynamic type system, this variable can contain a value of any type. This variable starts off set to `40`, which is, of course, a numeric value. We then add `2` to it, giving `42`. Then we change track slightly and make `answer` hold a string value. This is a common technique in a dynamic language, and it causes no syntax errors.

The JavaScript interpreter is also able to distinguish between the two uses of the `+` operator. The first use of `+` is numeric addition— adding `2` to `40`—whereas in the following line, the interpreter figures out from context that the developer meant string concatenation.

Let's try this trick again in Java using JShell:

```
jshell> var answer = 40;
answer ==> 40

jshell> answer = answer + 2;
answer ==> 42

jshell> answer = "What is the answer? " + answer;
|  Error:
|  incompatible types: java.lang.String cannot be converted to int
|  answer = "What is the answer? " + answer;
|           ^----------------------------^
```

Boom. Even though the exact same source code looked legal in both languages, Java's *static type system* prevented the final line from working. Java's `var` keyword does more than simply create the variable `answer`. As we learned in section 1.3, Java's `var` also inferred the type of this new variable from the right-hand side of the expression. We didn't

have to specify the type of `answer` explicitly, but Java's static type system assigns a type that never subsequently changes.

NOTE The key point here is that dynamic typing tracks type information with the *value* a variable contains (e.g., a number or a string), where static typing tracks the type with the *variable* definition instead.

Static typing can be a good fit for a compiled language because the type information is all about the variables, not the values in them. This allows reasoning about potential type system violations at compile time, before the code even has a chance to run.

Dynamically typed languages carry type information on the values held in variables. This provides a lot of flexibility but means type violations (e.g., "I thought this was a number, but it's a string") happen during execution. This can lead to more runtime errors, which can be harder and more expensive to debug than compile-time errors.

1.3 Imperative vs. functional languages

Java is a classic example of an imperative language. Imperative languages can be thought of as languages that model the running state of a program as mutable data and issue a list of instructions that transform that running state. Program state is thus the concept that has center stage in imperative languages.

Two main subtypes of imperative languages exist. Procedural languages, such as BASIC and FORTRAN, treat code and data as completely separate and have a simple code-operates-on-data paradigm. The other subtype is object-oriented (OO) languages, where data and code (in the form of methods) are bundled together into objects. The program state in an object-oriented system is the state of all the objects in the program. In OO languages, additional structure is imposed to a greater or lesser degree by metadata (such as class information). The differences between these subtypes, though, may not always be clear. C++, for instance, is explicitly intended to support both OO and procedural coding, and some later varieties of BASIC sport object-oriented features.

Functional languages take the view that computation itself is the most important concept. Functions operate on values, as in procedural languages, but instead of altering their inputs, functions are seen as acting like mathematical functions and return new values. Composing separate functions together in new and novel ways is also fundamental in this model.

As illustrated in figure 8.1, functions are seen as "little processing machines" that take in values and output new values. They don't have any state of their own, and it doesn't re-

ally make sense to bundle them up with any external state. The object- centered view of the world is somewhat at odds with the natural viewpoint of functional languages.

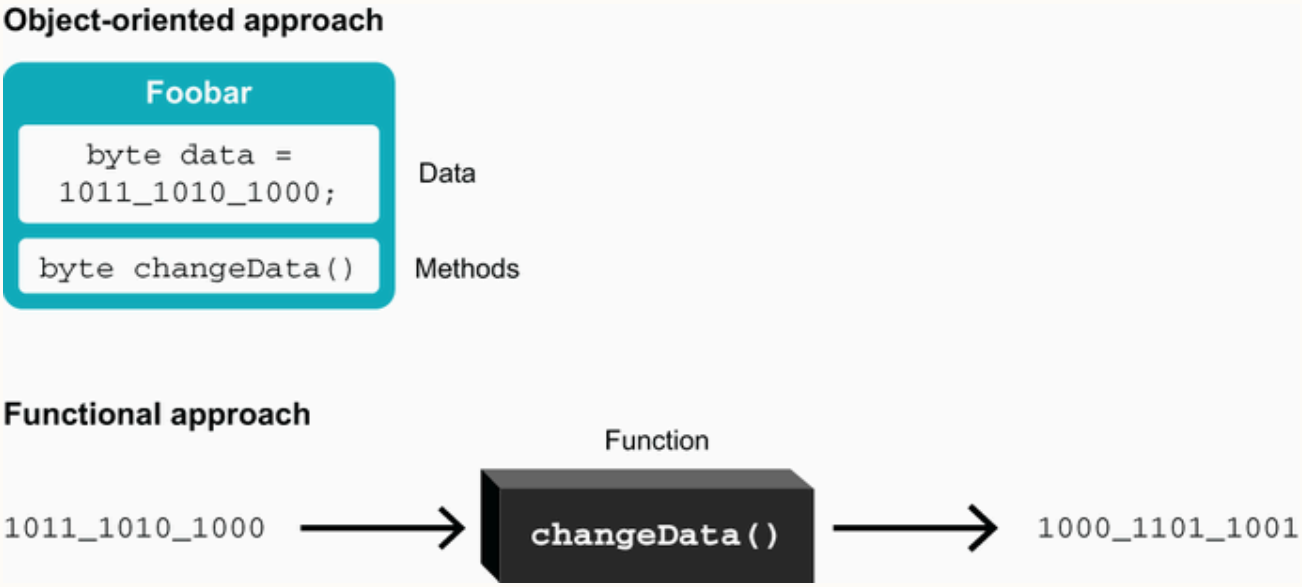


Figure 8.1 Imperative and functional languages

A key feature of functional languages is *first-class functions*—the ability to treat a function as a value, assigning it to variables, passing it to other functions, and even returning functions from other functions.

This is a great example of the feature spectrum we discussed earlier, because Java 8 added the lambda expression syntax, which enables Java programmers to treat functions as value. However, as a recent addition, the feature isn't used everywhere it could be in the platform, and older techniques for getting similar behavior, such as the `Runnable` and `Callable` interfaces, remain in use.

In the next two chapters we'll learn about different languages, and a key focus will be on how they support functional programming approaches. With Kotlin, we'll see how even an imperative language can be designed to smoothly support functional ideas. Then we'll look at Clojure, a much purer functional language that no longer centers on object orientation at all.

1.4 Reimplementation vs. original

Another important distinction between JVM languages is the division into those that are reimplementations of existing languages versus those that were specifically written to target the JVM. In general, languages that were specifically written to target the JVM provide a much tighter binding between their type systems and the native types of the JVM.

The following three languages are JVM reimplementations of existing languages:

- *JRuby* is a JVM reimplementation of the Ruby programming language. Ruby is a dynamically typed OO language with some functional features. It's basically interpreted on the JVM, but recent versions have included a runtime JIT compiler to produce JVM bytecode under favorable conditions.
- *Jython* was started in 1997 by Jim Hugunin as a way to use high-performance Java libraries from Python. It's a reimplementation of Python on the JVM, so it's a dynamic, mostly OO language. It operates by generating internal Python bytecode, then translating that to JVM bytecode. Sadly, the project has seen little activity since 2015 and supports only Python 2.7, not the current Python 3.
- *Rhino* was originally developed by Netscape and later the Mozilla project. It provided an implementation of JavaScript on the JVM and shipped up through JDK.
    - JDK 8 included a new JavaScript engine, *Nashorn* (The name "Nashorn" is a bit of a pun—it's the German word for "Rhino"), but the increasing pace of JavaScript language changes forced its deprecation with JDK 11 and removal in JDK 15. Although no JavaScript implementation will ship directly with future JDKs, both may still be found independently. (Rhino from Mozilla (**https://github.com/mozilla/rhino**) and Nashorn (**https://openjdk.java.net/projects/nashorn/**) as an independent OpenJDK project, which intends to live on and should be supported on future JDKs.

NOTE The earliest JVM language? The earliest non-Java JVM language is hard to pin down. Certainly, Kawa, an implementation of Lisp, dates to 1997 or so. In the years since then, we've seen an explosion of languages, to the point that it's almost impossible to keep track of them.

A reasonable guess at the time of writing is that at least 200 languages target the JVM. Not all can be considered to be active or widely used (and some are really very niche), but the large number indicates that the JVM is a very active platform for language development and implementation.

NOTE In the versions of the language and VM spec that debuted with Java 7, all direct references to the Java language have been removed from the VM spec. Java is now simply one language among many that run on the JVM—it no longer enjoys a privileged status.

The key piece of technology that enables so many different languages to target the JVM is the class file format, as we discussed in chapter 4. Any language that can produce a class file is considered a compiled language on the JVM.

Let's move on to discuss how polyglot programming came to be an area of interest for Java programmers. We'll explain the basic concepts and why and how to choose an alternative JVM language for your project.

8.2 Polyglot programming on the JVM

The phrase *polyglot programming on the JVM* was coined to describe projects using one or more non-Java JVM languages alongside a core of Java code. One common way to think about polyglot programming is as a form of separation of concerns. As you can see in figure 8.2, potentially three layers exist where non-Java technologies can play a useful role. This diagram is sometimes called the polyglot programming pyramid, and it's originally due to the work of Ola Bini (**https://olabini.com/blog/tag/polyglot/**).



Figure 8.2 The polyglot programming pyramid

Within the pyramid, dependencies run in one direction—the stable layer is relatively independent, the dynamic layer uses the stable, and domain-specific code can pull in from both layers below it.

Defining these layers in a given system isn't always easy; gray areas exist, and not all systems fit perfectly. However, it's a useful tool to identify the seams where different parts of the system have different needs and could benefit from different languages.

The stable layer contains core APIs and abstractions for your system. Type safety, thorough testing, and performance are all critical.

The dynamic layer uses the stable layer's abstractions to create a working system. This may include code such as how a system exposes itself over HTTP or interacts with other backend systems. Issues like compilation time and flexibility may make it worth considering a different language at the dynamic layer.

The domain-specific layer handles application-specific concerns such as presentation, customization of rules and processing, or CI/CD. This code is all about specific aspects of the application domain and may benefit from language choices that would be constraining in other layers.

NOTE Polyglot programming makes sense because different pieces of code have different lifetimes. A risk calculation engine in a bank may last for five or more years. JSP pages for a website could last for a few months. The most short-lived code for a startup could be live for just a few days. The longer the code lives, the closer to the stable layer of the pyramid it is. See table 8.1.

Table 8.1 Three layers of the polyglot programming pyramid

Name	Description	Examples
Domain-specific	Domain-specific language. Tightly coupled to a specific part of the application domain	Apache Camel, DSLs, Drools, web templating
Dynamic	Rapid, productive, flexible development of functionality	Clojure, Groovy, JRuby
Stable	Core functionality, stable, well-tested, performant	Java, Kotlin, Scala

As you can see, patterns occur in the layers—the statically typed languages tend to gravitate toward tasks in the stable layer. Conversely, the technologies intended for more of a specific purpose tend to be well-suited to roles at the top of the pyramid.

Let's dig a little deeper to look at why Java isn't the best choice for everything in the pyramid. We'll begin by discussing why you should consider a non-Java language, and then we'll cover some of the major criteria to look at in choosing a non-Java language for your project.

8.2.1 Why use a non-Java language?

Java's nature as a general-purpose, statically typed, compiled language provides many advantages. These qualities make it a great choice for implementing functionality in the stable layer. But these same attributes become a burden in the upper tiers of the pyramid, as described here:

- Recompilation is laborious.
- Static typing can be inflexible.

- Deployment is a heavyweight process.
- Java's syntax can be rigid and isn't a natural fit for producing DSLs.

The recompilation and rebuild time of a Java project often reaches the 90 seconds to two-minute mark. This is a long enough to seriously break a developers' flow, and it's a bad fit for developing code that may live in production for only a few weeks.

Java's rigid syntax

The Java language has a very rigid grammar. The fundamental language components are the keywords that are supplied. You cannot "make up new syntax" or create any new form that could be mistaken for a keyword.

The programmer can create new classes, and the capabilities of those classes consist of storing state in fields and calling methods on classes or objects. However, this is as far as it goes—the programmer cannot create anything that resembles a control structure. In other words, a field access will always look like the following:

```
anObject.someField
AClass.someStaticField
```

And a method call will always look like this:

```
anObject.someMethod(params)
AClass.someStaticMethod(params)
```

In Java, the method parameters are never optional (unlike in some other languages, such as Kotlin), so even the distinction between field access and methods calls cannot be blurred. For example, we cannot create constructs that look like keywords. For instance, we'd like to be able to create a `when` that looks like this:

```
when(value) {
  // action to be taken
}
```

But the best we can do is something like this:

```
import static when.When.when;
```

```
...

when(value, () -> {
  // action to be taken
});
```

This lack of redefinable syntax also shows up when trying to use Java to make DSLs. We'll see how our non-Java languages handle this issue in the next two chapters.

Overall, one pragmatic solution is to play to Java's strengths. Take advantage of its rich API and library support to do the heavy lifting for the application down in the stable layer.

Even within the stable layer you may find there are reasons a language other than Java may be desirable, such as the following:

- Java's verbosity can be off-putting for some developers, and it can hide certain classes of bugs.
- Although Java increasingly supports functional programming, limits in the ease of applying some patterns remain.
- Other languages present alternatives for concurrency that are not present in Java (coroutines in Kotlin, agents in Clojure).

NOTE Even if you choose another language to use in your stable layer for features it supports, you shouldn't throw out working code just to rewrite it so the languages match. Consider using the new language for new features or for low-risk areas we'll talk about identifying later in this chapter.

At this point, you may be asking yourself, What type of programming challenges fit inside these layers? Which languages should I choose? A well-grounded Java developer knows that there is no silver bullet, but we do have criteria to consider when evaluating your choices.

2.2 Up-and-coming languages

For the rest of the book, we've picked two languages that we see having great potential longevity and influence. These are two of the languages on the JVM (Kotlin and Clojure) that already have well-established mind share among polyglot programmers. Why are these languages gaining traction? Let's look at each in turn.

Kotlin

Kotlin is an imperative, static-typed, OO language from JetBrains (makers of IntelliJ IDEA). It aims to address the most common complaints about Java, while keeping a familiar development environment. Kotlin is a compiled language and has a high degree of compatibility beyond the basics that just running on the JVM provides.

Key features in Kotlin include concise syntax, `null` safety, extremely strong interoperability with Java code, and coroutines—an alternate concurrency mechanism to Java's traditional threading model. A number of features from Kotlin have found their way back into Java in recent releases, confirming the value those changes presented to developers.

Although it has established itself as a key JVM language option in multiple areas, Kotlin has shown particular success in mobile, with the Android platform adopting it as the recommended language in 2019. Kotlin is also supported for Gradle build scripting at the same level as Groovy. It's also been embraced by many other frameworks, such as Spring. The wide range of convenience and safety improvements for developers are worth considering regardless of where your JVM is running. Chapter 9 gives an introduction to Kotlin.

Kotlin will be used in chapter 11 as the primary scripting language for Gradle builds. We will also revisit it to demonstrate some unique approaches to functional programming in chapter 15 and concurrent programming (coroutines) in chapter 16.

Clojure

Clojure, designed by Rich Hickey, is a language from the Lisp family. It inherits many syntactic features (and lots of parentheses) from that heritage. It's a dynamically typed, functional language, as is usual for Lisps. It's a compiled language but usually distributes code in source form for reasons we'll see later. It also adds a significant number of new features (especially in the arena of concurrency) to its Lisp core.

Lisps are usually seen as experts-only languages. Clojure is somewhat easier to learn than other Lisps, yet it still provides the developer with formidable power (and also lends itself very nicely to the test-driven development style). But it's likely to remain outside the mainstream, being primarily used by enthusiasts and for specialized jobs (e.g., some financial applications find its combination of features very appealing).

Clojure is best thought of as sitting in the dynamic layer but, due to its concurrency support and other features, can be seen as capable of performing many of the roles of a sta-

ble layer language. Chapter 10 provides an introduction to Clojure.

We will use Clojure extensively in learning more about functional programming beyond what Java can do in chapter 15. It will also feature in introducing the actor model, a powerful alternative in concurrent programming in chapter 16.

2.3 Languages we could have picked but didn't

As noted earlier, a huge variety of languages exist that we could look at. Here's a little more about a few other contenders that may be practical for you to look at more deeply yourself.

Groovy

The Groovy language was invented by James Strachan in 2003. It's a dynamic, compiled language with syntax very similar to Java's but more flexible. It's widely used for scripting and testing. It was the original language used by the Gradle build tool and is used for configuring Jenkins, an extremely common CI/CD tool. It's often the first non-Java language that developers or teams investigate on the JVM. Groovy can be seen as sitting in the dynamic layer and is also known for being great for building DSLs.

We've chosen not to cover Groovy in more detail because it has seen declining mindshare in the prototyping and application use cases in the face of improving frameworks and other languages.

Scala

Scala is an OO language that also supports many aspects of functional programming. It traces its origins to 2003, when Martin Odersky began work on it in an academic setting, following his earlier projects related to generics in Java. It's a statically typed, compiled language like Java, and it performs a large amount of type inference, so it often has the feel of a dynamic language.

Scala learned a great deal from Java, and its language design "fixes" several common annoyances with Java. However, Scala has ended up with a very large set of features, though, and a much more advanced type system compared to Java's.

It can be complicated to program and is not easy to learn thoroughly. As such we've chosen to focus on Kotlin for developers who just want improvements to the state of the Java language.

GraalVM

Oracle Labs have produced GraalVM (**https://www.graalvm.org/**), which they describe as a polyglot virtual machine and platform that is partly derived from Java and JVM codebases. The current release includes the capability to run Java and other JVM languages (as bytecode) as well as support for JavaScript and LLVM bitcode (the intermediate representation from the popular LLVM compiler), with beta support for Ruby, Python, R, and WASM.

The overall platform comprises the following components:

- Java HotSpot VM
- A Node.js JavaScript runtime environment
- LLVM runtime to execute LLVM bitcode
- Graal—a JIT compiler written in Java
- Truffle—a toolkit and API for building language interpreters
- SubstrateVM—a lightweight execution container for native images

Within a GraalVM project, languages can be very freely bridged to each other, and the aim is to allow components implemented in different technologies to be combined and used in a single application process. This is a very different approach to polyglot programming, but it is close enough to the subject of interest that we wanted to at least mention it.

Non-JVM languages

This chapter focuses on alternative languages that run on the JVM. It's worth admitting, though, that sometimes the polyglot programmer may have a reason that part of their system needs to leave the JVM behind entirely.

Examples of technologies which have broader support outside the JVM follow:

- Native system code (C, Go, or Rust)
- Machine learning (Python)
- Run in a user's web browser (JavaScript)

Although JVM-based approaches exist for many of these, it's worth taking stock of the maturity of alternatives and the composition of our team before trying to keep every line of code entirely on the JVM.

Now that we've outlined some possible choices, let's discuss the issues that should drive your decision of which language to choose.

8.3 How to choose a non-Java language for your project

Once you've decided to experiment with non-Java languages in your project, you need to identify which parts of your project naturally fit into the stable, dynamic, or domain-specific layers. Table 8.2 highlights tasks that might be suitable for each layer.

Table 8.2 Project areas suited for domain-specific, dynamic, and stable layers

Name	Example problem domains
Domain-specific Domain-specific areas often benefit from readability by experts who may not know Java. Software life cycle tooling also frequently has domain-specific languages and configuration.	Build, continuous integration, continuous deployment Dev-ops Business rules modeling
Dynamic Dynamic layers of the system may benefit from greater flexibility and speed of development available in other languages. This may be especially true for tooling that is internally facing (testing and administrative).	Rapid web development Prototyping Interactive administrative and user consoles Scripting Testing
Stable Stable layer code expresses a system's core abstractions. More rigorous type safety and testing are worth the additional developer overhead.	Concurrent code Application containers Core business functionality

As you can see, a wide range of use cases for alternative languages exists. But identifying a task that could be resolved with an alternative language is just the beginning. You next need to evaluate whether using an alternative language is appropriate. Here are some useful criteria that we take into account when considering technology stacks:

- Is the project area low-risk?
- How easily does the language interoperate with Java?
- What tooling support (e.g., IDE support) is there for the language?
- How steep is the learning curve for this language?
- How easy is it to hire developers with experience in this language?

Let's dive into each of these areas so you get an idea of the sorts of questions you need to be asking.

3.1 Is the project area low-risk?

Let's say you have a core payment-processing rules engine that handles millions of transactions a day. This is a stable piece of Java software that has been around for over seven years, but there aren't a lot of tests and plenty of dark corners are present in the code. The core of the payment-processing engine is clearly a high-risk area to bring a new language into, especially when it's running successfully and there's a lack of test coverage and of developers who fully understand it.

But there's more to a system than its core processing. For example, this is a situation where better tests would clearly help. Kotlin has a number of good options, including the Spek framework (**https://www.spekframework.org/**) and Kotest (**https://kotest.io**), which leverage the language to enable clear, readable specifications without the typical JUnit boilerplate. Or perhaps your rules engine would benefit from *property testing*, where tests are written to validate conditions against generated inputs and Clojure's `test.check` (**https://clojure.org/guides/test_check_beginner**) would be a valuable tool in the mix.

Or suppose you need to build a web console so that the operations users can administer some of the noncritical static data behind the payment-processing system. The development team members already know Struts and JSF but don't feel any enthusiasm for either technology. This is another low-risk area to try out a new language and technology stack. Spring Boot with Kotlin would be one obvious choice.

By focusing on a limited pilot in an area that is low-risk, there's always the option of terminating the project and porting to a different delivery technology without too much disruption if the new technology stack isn't a good fit.

3.2 Does the language interoperate well with Java?

You don't want to lose the value of all of that great Java code you've already written! This is one of the main reasons organizations are hesitant to introduce a new program-

ming language into their technology stack. But with alternative languages that run on the JVM, you can turn this on its head, so it becomes about maximizing your existing value in the codebase and not throwing away working code.

Alternative languages on the JVM are able to cleanly interoperate with Java and can, of course, be deployed on a preexisting environment. This is especially important to avoid impacting whoever owns deployment, whether a production management team or DevOps folks in your own team. By using a non-Java JVM language as part of your system, you retain your organization's operational expertise, which can help alleviate worries and reduce risk around supporting the new solution.

NOTE DSLs are typically built using a dynamic (or, in some cases, stable) layer language, so many of them run on the JVM via the languages that they were built in.

Some languages interoperate with Java more easily than others. We've found that most popular JVM alternatives (such as Kotlin, Clojure, JRuby, Groovy, and Scala) all have good interoperability with Java (and for some of the languages, the integration is excellent, almost completely seamless). If you're a really cautious shop, it's quick and easy to run a few experiments first and make certain that you understand how the integration can work for you.

Let's take Kotlin, for example. You can import Java packages directly into its code via the familiar import statement. From here you could easily write a small Kotlin script, or even use the interactive Kotlin shell, to poke at your Java model objects and see what the interoperation surfaces will look like. We'll talk specifically about Java interoperability in the language chapters coming up next.

3.3 Is there good tooling and test support for the language?

Most developers underestimate the amount of time they save once they've become comfortable in their environment. Their powerful IDEs and build and test tools help them to rapidly produce high quality software. Java developers have benefited from great tooling support for years, so it's important to remember that other languages may not be at quite the same level of maturity.

Some languages (such as Kotlin) have had longstanding IDE support for compiling, testing, and deploying the end result. Other languages may have tooling that hasn't matured as fully.

A related issue is that when an alternative language has developed a powerful tool for its own use (such as Clojure's awesome Leiningen build tool), the tool may not be well

adapted to handle other languages. Therefore, the team will need to think carefully about how to divide up a project, especially for deployment of separate but related components.

3.4 How hard is the language to learn?

It always takes time to learn a new language, and that time only increases if the paradigm of the language isn't one that your development team is familiar with. Most Java development teams will be comfortable picking up a new language if it's object-oriented with a C-like syntax (such as Kotlin).

It gets harder for Java developers as they move further away from this paradigm. At the extreme of the popular alternative languages, a language such as Clojure can bring incredibly powerful benefits, but it can also represent a significant retraining requirement for development teams as they learn Clojure's functional nature and Lisp syntax.

One alternative is to look at the JVM languages that are reimplementations of existing languages. Ruby and Python are well-established languages, with plenty of material available for developers to use to educate themselves. The JVM incarnations of these languages could provide a sweet spot for your teams to begin working with an easy-to-learn non-Java language.

3.5 Are there lots of developers using this language?

Organizations have to be pragmatic; they can't always hire the top 2% (despite what their advertising might say), and their development teams will change throughout the course of a year. Some languages, such as Kotlin or Scala, are becoming well-established enough that there is a pool of developers to hire from. But a language such as Clojure may present more difficulties. Managers may push back against using something out of the ordinary from concern for creating an unmaintainable codebase they'll have difficulty hiring for.

NOTE A warning about the reimplemented languages: many existing packages and applications written in Ruby, for example, are tested only against the original C-based implementation. There may be problems when trying to use them on top of the JVM. When making platform decisions, you should factor in extra testing time if you're planning to leverage an entire stack written in a reimplemented language.

Again, the reimplemented languages (JRuby, Jython, and so on) can potentially help here. Few developers may have JRuby on their CV, but because it's just Ruby on the JVM,

there's actually a large pool of developers to hire from—a Ruby developer familiar with the C version can learn the differences induced by running on the JVM very easily.

We now have a set of questions to ask when choosing an alternative language and an overview of some available options. At this point, it's worth a deeper understanding of how the JVM supports multiple languages. This peek reveals the roots of some design choices and limitations in alternative languages on the JVM.

8.4 How the JVM supports alternative languages

A language can run on the JVM in two possible ways:

- Have a source code compiler that emits class files. Kotlin and Clojure both function in this way.
- Have an interpreter that is implemented in JVM bytecode. JRuby is implemented in this way.

In both cases, it's usual to have a runtime environment that provides language-specific support for executing programs. Figure 8.3 shows the runtime environment stack for Java and for a typical non-Java language.

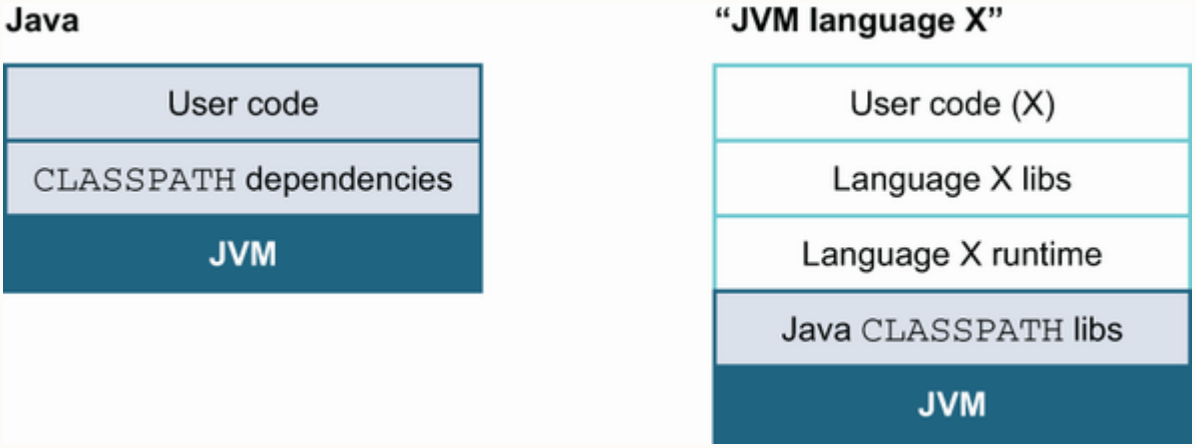


Figure 8.3 Non-Java language runtime support

These runtime support systems vary in complexity, depending on the amount of hand-holding that a given non-Java language requires at runtime. In almost all cases, the runtime will be implemented as a set of JARs or modules that an executing program needs to have on its classpath. In the interpreted case, the interpreter will bootstrap as program execution starts and then read in the source file to be executed.

4.1 Performance

One question developers often ask about different languages is, How do they perform relative to each other? Although superficially attractive, this question is not simple to answer and, in fact, is not actually all that meaningful.

As we saw in chapter 7, the well-grounded developer knows that performance is driven by measurement. Measurement is done on individual programs, not on the abstract notion of a programming language. Treat any claim that language X "performs better" than Y without accompanying, reliable data as suspect.

However, in practice, some overall performance characteristics of a JVM language can broadly be determined by how the language is implemented. A compiled language is just bytecode at runtime and will be JIT-compiled in the same way as Java is. An interpreted language will have very different performance behavior because the code that gets JIT-compiled is the interpreter, and not the program itself.

NOTE Some languages (e.g., JRuby) have a hybrid strategy—they have an interpreter for scripts but can also dynamically compile individual source methods to JVM bytecode, which can then be compiled to machine code by the JVM's JIT compilers.

In this book, our focus is on compiled languages. The interpreted languages—such as Rhino—are mentioned for completeness, but we won't spend too much time on them. We, therefore, expect that performance will be broadly similar between the languages that we're considering. For a more detailed answer, you should undertake a detailed analysis of a specific program or workload.

In the rest of this section, we'll discuss the need for runtime support for alternative languages (even for compiled languages) and then talk about compiler fictions—language-specific features that are synthesized by the compiler and that may not appear in the low-level bytecode.

4.2 Runtime environments for non-Java languages

One simple way to measure the complexity of the runtime environment that a particular language requires is to look at the size of the JAR files that provide the implementation of the runtime. Using this as a metric, we can see that Clojure is a relatively lightweight runtime, whereas JRuby is a language that requires more support.

This isn't a completely fair test, because some languages bundle much larger standard libraries and additional functionality into their standard distributions than others.

However, it can be a useful (if rough) rule of thumb.

In general, the purpose of the runtime environment is to help the type system and other aspects of the non-Java language achieve the desired semantics. Alternative languages don't always have exactly the same view as Java about basic programming concepts.

For example, Java's approach to OO isn't universally shared by other languages. In Java, all objects that are instances of a particular class all have exactly the same set of methods on them, and that set is fixed at compile time. In Ruby, on the other hand, an individual object instance can have additional methods attached to it at runtime that were not known when the class was defined and that aren't necessarily defined on other instances of the same class.

NOTE The `invokedynamic` bytecode was, in fact, originally added to the JVM to facilitate the efficient implementation of these types of language features.

This ability to dynamically add methods (which is somewhat confusingly called "open classes") needs to be replicated by the JRuby implementation. This is possible only with some advanced support from the JRuby runtime.

4.3 Compiler fictions

Certain language features are synthesized by the programming environment and high-level language and aren't present in the underlying JVM implementation. These are referred to as *compiler fictions*.

NOTE It helps to have some knowledge of how these features are implemented; otherwise, you can find your code running slowly or, in some cases, even crashing the process. Sometimes the environment has to do a lot of work to synthesize a particular feature.

Other examples in Java include checked exceptions and inner classes (which are always converted to top-level classes with specially synthesized access methods if necessary, as shown in figure 8.4). If you've ever looked inside a JAR file (using `jar tvf`) and seen a load of classes with `$` in their names, these are the inner classes unpacked and converted to "regular" classes.

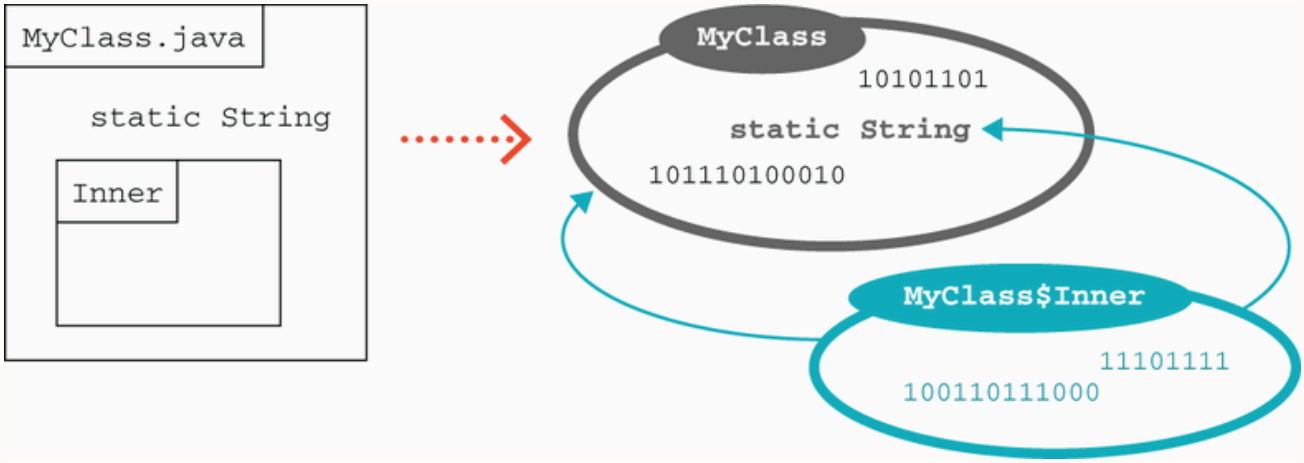


Figure 8.4 Inner classes as a compiler fiction

Alternative languages also have compiler fictions. In some cases, these compiler fictions even form a core part of the language's functionality.

In section 8.2, we introduced the key concept of *first-class functions* in functional programming—that functions should be values that can be put into variables. All the non-Java languages in part 3 of this book supported this feature long before Java added lambda expressions. How did they accomplish this when the JVM handles only classes as the smallest unit of code and functionality?

The original solution to this discrepancy between source code and JVM bytecode is to remember that objects are just bundles of data along with methods to act on that data. Imagine an object with no state and just one method—for example, a simple anonymous implementation of Java's `Callable`. It wouldn't be at all unusual to put such an object in a variable, pass it around, and then invoke its `call()` method later, like this:

```
Callable<String> myFn = new Callable<String>() {
    @Override
    public String call() {
        return "The result";
    }
};

System.out.println(myFn.call());
```

NOTE The `myFn` variable in this example is an anonymous type, so it will show up after compilation as something like `NameOfEnclosingClass$1.class`. The class numbers start at 1 and go up for each anonymous type the compiler encounters. If they're dynamically created, and there are a lot of them (as sometimes happens in languages like JRuby), this can place pressure in the off-heap memory where the definitions of classes are stored.

Java lambda expressions don't actually use this anonymous type approach but instead are built on a general JVM feature called `invokedynamic`, which we'll discuss in detail in chapter 17. Alternate languages are moving from their specialized implementations to use `invokedynamic`, too. It's an interesting case of compiler fictions influencing the development of platform realities.

For another example, in the next chapter we will meet Kotlin's *data classes*— a language feature that helps to lower the amount of typing and *ceremony* required when declaring a class that is "just a dumb bunch of fields." In Kotlin as it exists today, this is a compiler fiction, but Java 17 has added a feature called *records*, which may eventually provide an alternative basis for Kotlin to build data classes upon.

Summary

- Alternative languages on the JVM have come a long way to offer better solutions than Java for certain problems.
- Languages can be classified in different ways (static versus dynamic, imperative versus functional, and compiled versus interpreted), which can assist in picking the right language for the right task.
- Polyglot programming is often separated into three layers: stable, dynamic, and domain-specific. Java and Kotlin are best for the stable layer of software development. Clojure may be more suited to tasks in the dynamic or domain-specific realms.
- Core business functionality of an existing production application is almost never the correct place to introduce a new language. Choose a low-risk area for your first deployment of an alternative language.
- Teams and projects have unique characteristics that will impact language choice. There are no universal right answers here.

Previous chapter
‹ Part 3. Non-Java languages on the JVM

Next chapter
9 Kotlin ›